

AI Daily Academy

day 003 2026-08-14

通勤用音声教材の文字版 - 最新情報の参照元は末尾に掲載

Week 1 「AIの全体像」

Day 3 「Embeddingとベクトル」

1. オープニング

この音声はAI生成音声でお届けします。

おはようございます。AI Daily Academy、Week 1、Day 3です。

今日のテーマは、「Embedding、エンベディング」と「ベクトル」です。

昨日までに、生成AIとLLM、そしてトークンとコンテキストについて学びました。LLMは言葉をトークンという小さな単位に分け、限られたコンテキスト、つまり作業用の記憶の中で、次に来るもっともらしい言葉を予測して文章を作ります。

では、AIは「意味が近い文章」をどう見つけるのでしょうか。

たとえば、「出張の交通費を精算したい」と、「新幹線代の申請方法を知りたい」。使われている言葉は少し違いますが、人間なら似た話だとわかりますよね。

その「意味の近さ」を、コンピュータが計算できる形に変える技術が、今日の主役、Embeddingです。

通勤中は、すべてを一度で覚える必要はありません。今日はまず、「言葉を座標に変える」「近い座標は意味も近いことが多い」「その近さを使って必要な情報を探す」。この3つを持ち帰れば十分です。

2. 前回の復習

まず、Day 2の復習です。

LLMが文章を扱うとき、最初から人間のように単語や文の意味を丸ごと理解しているわけではありません。文章をトークンに分割し、その並びをもとに、次のトークンを予測します。

ここで大切なのがコンテキストです。コンテキストは、モデルがその瞬間に参照できる情報の範囲です。会話の履歴、与えられた指示、添付された資料の一部などが入ります。

ただし、会社の規程集、数年分の議事録、商品カタログ、問い合わせ履歴など、すべてを毎回コンテキストに入れるのは現実的ではありません。量が多すぎますし、関係のない情報まで混ぜると、AIが大事な部分を見失うこともあります。

そこで必要になるのが、「大量の情報の中から、今の質問に意味が近い部分だけを選び出す」仕組みです。

その土台にあるのがEmbeddingです。

昨日のキーワードが「トークン」と「コンテキスト」だったなら、今日のキーワードは「意味を数値の並びに変えること」、そして「距離で近さを測ること」です。

3. 今日の基礎講義

Embeddingを一言で言うと、**文章、単語、画像、音声などの特徴を、多数の数値が並んだベクトルに変換すること**です。

ベクトルという言葉を知ると、数学を思い出して身構えるかもしれません。でも、ここでは難しく考えなくて大丈夫です。

たとえば、東京駅の位置を、地図上の座標で表すことを考えてみましょう。

「東にどれくらい」「北にどれくらい」と数字で表せば、場所同士の距離を計算できます。

Embeddingも、考え方は似ています。

ただし、表すのは地図上の場所ではありません。表すのは、言葉や文章が持つ意味の特徴です。

たとえば、次の三つの文を考えてみます。

「猫が窓辺で寝ている」

「子猫が日なたで眠っている」

「法人税の申告期限を確認したい」

最初の二つは、使っている語が完全には同じではありません。それでも、猫、寝る、穏やかな場面、といった意味が近い。三つ目は税務の話なので、かなり離れています。

Embeddingモデルは、大量のデータから学んだパターンを使って、それぞれの文を数百次元から数千次元程度の数字の列に変換します。たとえば、三次元ではなく、千次元の空間に置くイメージです。

千個の数字を人間が見て意味を読むことはできません。ですがコンピュータは、その数字の並び同士がどれくらい似ているかを計算できます。

つまり、Embeddingは辞書ではありません。

「猫」という文字列があるから猫、と機械的に照合するだけのものでもありません。文全体の文脈を踏まえ、数値の配置として意味の近さを表そうとする技術です。

ここで重要な注意点があります。

Embeddingは「意味を完全に理解した証明」ではありません。また、近いベクトルだからといって、内容が正しいとは限りません。

Embeddingが得意なのは、正誤判定ではなく、候補を意味の近さで並べることです。

言い換えると、「答えを保証する技術」ではなく、「探すべき資料の候補を絞り込む技術」です。

この違いは、実務ではとても大切です。

4. 仕組み・応用

では、ベクトル同士の近さはどう測るのでしょうか。

代表的なのが、コサイン類似度です。

名前は少し難しそうですが、イメージは単純です。二つのベクトルが「どちらの方向を向いているか」を比べる方法です。

同じ方向に近ければ、類似度は高くなります。

直角に近ければ、あまり関係がありません。

反対方向に近ければ、似ていないとみなされます。

ここで、ベクトルの長さではなく方向を見ることに意味があります。

短い文章と長い文章でも、伝えている中心的な意味が似ていれば、近い方向を向くことがあります。

たとえば、「休暇申請のやり方を教えて」と、「有給休暇を取得する手続きは？」は、表現が違っていても似た方向に配置されやすい。

一方で、「休暇中の自動返信メールの文面を作って」は、「休暇」という単語を含みますが、知りたいことは申請手続きとは別です。Embeddingはこうした違いも、ある程度は捉えようとします。

ただし、ここで「ある程度」という言葉を忘れないでください。

専門用語、社内略語、固有名詞、否定表現、数字、日付、細かな条件では、意味の近さだけでは不十分なことがあります。

たとえば、「返金できる」と「返金できない」は、含まれる単語がほぼ同じです。ベクトルも近くなりやすいことがあります。しかし実務上は、正反対の結論です。

このため、検索システムではEmbeddingだけに頼り切らず、キーワード検索、日付、部署、文書種別などの条件も一緒に使います。

Embeddingの代表的な応用は、意味検索です。

従来のキーワード検索では、「経費精算」という語が含まれる文書を探すのは得意です。しかし利用者が「立替分をあとで戻してもらうには」と尋ねた場合、「経費精算」という語がなければ見つけにくいことがあります。

意味検索では、質問をEmbeddingに変換し、登録済みの文書のEmbeddingと比べます。

そして、もっとも意味が近い文書、あるいは文書の一部を上位に出します。

この「文書の一部」がポイントです。

長い文書を丸ごと一つのベクトルにすると、情報が混ざりすぎます。就業規則全体を一つの塊にすると、休暇、残業、福利厚生、懲戒など、さまざまな話題が一つのベクトルに押し込まれます。

そこで普通は、文書を適度な長さに分けます。これをチャンク分割、あるいはチャンキングと呼びます。

たとえば、見出しごと、段落ごと、あるいは数百トークンごとに区切ります。

それぞれのチャンクに、文書名、作成日、部署、公開範囲、見出しなどのメタデータも付けます。

質問が来たら、次の流れです。

まず、質問をEmbeddingにします。

次に、ベクトルデータベースや検索インデックスから、近いチャンクをいくつか探します。

必要なら、部署が人事である、最新版である、閲覧権限がある、といった条件で絞ります。

その後、上位候補をより精密なモデルで並べ替えることがあります。これをリランキングと呼びます。

最後に、本当に関連がある資料だけをLLMのコンテキストに渡し、要約や回答を作らせます。

この全体の流れが、RAG、ラグと呼ばれる仕組みの中心です。

RAGは、Retrieval-Augmented

Generationの略で、日本語では「検索拡張生成」と訳されることがあります。

重要なのは、RAGは単に「AIに社内資料を読ませること」ではない、という点です。

正確には、質問に応じて関連資料を検索し、その資料を根拠として生成AIに回答させる設計です。

Embeddingは、このRAGの「検索」の中心で働きます。

もう一つ、実装上の重要な言葉があります。近似最近傍探索です。

文書が100件なら、質問のベクトルと100件すべてを比べても問題ありません。

しかし、文書が100万件、1000万件になると、毎回すべてと比べるのは重くなります。

そこで、「完全に厳密な近さの順番」ではなく、「十分に近い候補を高速に見つける」ための索引を作ります。これが近似最近傍探索です。

ここから実務の感覚を一つ。

検索が速いことは大切ですが、速さだけでは足りません。検索結果の質、権限の正しさ、文書の更新、回答の根拠表示、ログの確認まで含めて、初めて業務で使える仕組みになります。

Embeddingは魔法の検索エンジンではありません。

データ整備、検索設計、評価、運用を支える部品です。

5. 身近な具体例

身近な例で考えてみましょう。

まず、写真アプリです。

「海」「犬」「桜」などで写真を探せる機能があります。これは、画像そのものをベクトルに変え、入力された言葉や別の画像と意味的に近いものを探す考え方と相性がよい機能です。

次に、動画や音楽のおすすめです。

あなたが視聴した作品、途中で止めた作品、高く評価した作品、似た行動をした他の利用者の傾向などを、さまざまな特徴として扱います。そのうえで、好みに近い候補を順位付けします。Embeddingが唯一の方法ではありませんが、「似ているものを数値空間で探す」という発想は共通しています。

通販サイトもわかりやすい例です。

「通勤用で、雨に強くて、ノートパソコンが入るバッグ」と探すとしたら。単純なキーワードの一致だけでは、利用者の意図を取りこぼします。「撥水」「PC収納」「ビジネスリュック」といった別表現も含め、意味的に近い商品や説明文を見つける必要があります。

そして、社内検索です。

「在宅勤務の日に使える経費の範囲を知りたい」と質問したとき、規程の本文には「通信費」「在宅勤務手当」「業務上必要な費用」と書かれているかもしれません。質問の言い方と規程の言い方が違って、関連する箇所を拾える可能性がある。これが意味検索の便利さです。

ただし、規程のような場面では、検索結果をそのまま答えに はいけません。

最終的には、該当する規程の原文、適用日、対象者、例外条件を確認する必要があります。

Embeddingは「探す」のを助けます。

ルールを決めたり、責任を負ったりするのは、人と業務プロセスです。

6. 実務での実用例

ここからは、業務でどう使うかを、三つの場面で見えていきましょう。

一つ目は、社内ナレッジ検索です。

営業、総務、法務、開発、カスタマーサポートなど、どの部署にも資料があります。しかし資料が増えるほど、「存在するのに見つからない」という問題が起きます。

ここでEmbeddingを使うと、資料の表現が質問と異なっても、候補を出しやすくなります。

たとえば、営業担当が「競合との違いを短く説明したい」と検索したとき、製品比較表、過去の提案書、営業トーク例、FAQなどを意味に近い順に出せます。

ただし、提案書には古い価格や古い機能が含まれているかもしれません。

そのため、メタデータで「最新版」「対象製品」「公開部署」「更新日」を持たせ、検索時に絞る必要があります。

二つ目は、問い合わせ対応です。

顧客からの問い合わせは、同じ内容でも表現がばらばらです。

「ログインできない」

「パスワードを忘れた」

「認証コードが届かない」

「アカウントに入れない」

これらは似ている部分もありますが、必要な対応は違う場合があります。

そこで、過去の問い合わせ、解決手順、ヘルプ記事をEmbeddingで検索し、近い事例を担当者に提示します。さらに、LLMが回答文の下書きを作る。人が確認して送信する。こうすると、対応速度と品質のばらつきを改善しやすくなります。

三つ目は、文書の分類と重複検知です。

新しく登録された契約書、報告書、障害報告、議事録などをEmbeddingに変換すると、似た文書を見つけたり、テーマごとにまとめたりできます。

たとえば、毎週届く顧客の要望をベクトル化しておけば、「今週は価格に関する不満が増えた」「特定機能への要望がまとまっている」といった傾向を把握する手がかりになります。

ただし、ここでも数字には注意です。

「月額1万円」と「月額10万円」は文章の意味が近く見えても、ビジネス上の差は大きい。金額、日付、契約条件、型番、法的表現は、別途ルールで確認する設計が必要です。

Embeddingが向いているのは、曖昧な言い換えや、大量データからの候補探しです。

厳密な一致が必要な場面では、構造化データ、フィルター、ルール、承認フローと組み合わせます。

7. 実装例

ここでは、「社内の経費規程を検索して答える小さなアシスタント」を例に、実装の全体像を見てみましょう。

まず、準備するものは四つです。

一つ、検索対象の文書。

二つ、文書を適切な単位に分ける処理。

三つ、各チャンクをベクトルに変換するEmbeddingモデル。

四つ、ベクトルを保存して近いものを探せる検索基盤です。

最初の工程は、文書の整形です。

PDFや社内Wikiから文章を取り出します。このとき、見出し、表、注記、改定日、文書名をできるだけ保持します。PDFの読み取り精度が低いと、検索以前にデータが壊れます。ここは地味ですが重要です。

次に、チャンク分割です。

おすすめは、単純に文字数だけで切るより、見出しや段落の意味を尊重することです。たとえば「国内出張旅費」「海外出張旅費」「宿泊費の上限」のような見出し単位で分けると、検索後に引用する内容もわかりやすくなります。

そして、各チャンクをEmbeddingに変換して保存します。

保存時には本文だけでなく、文書名、見出し、改定日、対象部署、閲覧権限、元のリンクなどを一緒に持たせます。

質問が来たら、まず質問をEmbeddingにします。

そのベクトルで近いチャンクを、たとえば上位5件、検索します。

次に、検索結果が本当に質問に答えているかを確認します。必要ならキーワード検索も混ぜます。

そして、関連チャンクだけをLLMに渡し、「根拠にないことは推測しない」「参照した規程名と見出しを示す」と指示して回答を作らせます。

処理の流れを、短い疑似コードで表すと、こうなります。ここは読み上げのために暗記する必要はありません。

```
```text
```

```
質問をベクトル化する
```

```
近い文書チャンクを検索する
```

```
権限と更新日で絞る
```

```
上位候補を再順位付けする
```

```
根拠付きでLLMに回答させる
```

```
```
```

実装で失敗しやすいポイントも押さえましょう。

一つ目は、チャンクが大きすぎることです。

検索には当たっても、LLMに渡す情報がぼやけます。

二つ目は、チャンクが小さすぎることです。

条件と例外が別々のチャンクに分かれ、「原則だけを答えて例外を落とす」危険があります。

三つ目は、評価をしないことです。

実装したら、実際に使われる質問を50件、100件と集めます。そして、「正しい資料が上位に出たか」「古い資料を出していないか」「権限外の資料を出していないか」「回答に根拠を表示できたか」を確認します。

四つ目は、Embeddingモデルの変更を軽く考えることです。

異なるEmbeddingモデルが作ったベクトルは、原則として同じ空間で比較できません。モデルを変えたら、既存文書も再度Embeddingし直す必要があることが多い。これは運用上の大切なポイントです。

最後に、セキュリティです。

社内文書を外部サービスに送る場合は、利用規約、保存期間、学習への利用有無、データの所在、アクセス制御を確認してください。検索結果を出す前に、利用者がその文書を見る権限を持つかを必ず判定します。

「検索できる」と「見せてよい」は別の問題です。

8. 今日のAI最新情報

ここでは、日本時間2026年8月14日時点で確認した、直近の公式発表を五つ紹介します。今日のテーマであるEmbeddingとのつながりも、一緒に見ていきましょう。

一つ目。8月13日、OpenAIは、GPT-5.6 Solを最大14倍高速に動かす「Ultrafast」の限定プレビューを発表しました。音声対応、顧客サポート、リアルタイム調査など、待ち時間が体験を左右する用途を想定しています。

Embeddingとの関係は、検索の速さと生成の速さを分けて考えることです。RAGでは、まずEmbedding検索で候補を取り出し、その後にLLMが回答を生成します。生成モデルの応答が速くなっても、検索が遅ければ全体の体験は遅いまです。逆に、検索と生成の両方を設計することが、リアルタイムAIでは重要になります。

二つ目。8月12日、OpenAIは企業でのAI活用に関する報告を公表し、先進的な企業ほど、AIエージェントを自社の文脈、ツール、繰り返し業務へ接続していると述べました。

今日のテーマとの関係は非常に直接的です。AIに「会社の文脈」を渡すには、社内資料や顧客情報を必要に応じて検索する仕組みが必要です。その入り口として、Embeddingによる意味検索が使われます。ただし、文脈をつなぐほど権限、監査、人による確認も重要になる、という点を忘れてはいけません。

三つ目。8月11日、OpenAIは、ChatGPT上の広告を日本、イギリス、メキシコ、ブラジル、韓国へ展開したと更新しました。OpenAIは、広告が回答そのものに影響しないこと、広告主に会話内容を渡さないことを方針として示しています。

Embeddingとの関係で注目したいのは、「関連性」と「中立性」を分けて設計する必要があることです。意味の近さを使えば、会話テーマに関連する情報を提示しやすくなります。しかし、関連することと、回答をゆがめないことは別です。検索でも広告でも、何を根拠に選んだのか、どこまで個人化するのか、どんなデータを使うのかという透明性が重要になります。

四つ目。8月10日、OpenAIは、防御目的のサイバーセキュリティ利用を対象にしたDaybreakの拡張と、GPT-5.6-Cyberを発表しました。脆弱性の発見、コードレビュー、インシデント対応などを想定し、利用者の審査や用途に応じたアクセス区分を設けています。

Embeddingとの関係は、セキュリティ運用での情報探索です。障害対応や脆弱性調査では、ログ、過去の障害報告、設定情報、手順書など、膨大な情報から関連する断片を素早く見つける必要があります。Embedding検索はその支援になり得ます。一方で、機密性の高い情報を扱うため、アクセス権や監査ログの設計は、一般的な社内検索以上に厳格であるべきです。

五つ目。8月7日、Anthropicは、Fable 5の生物学分野における安全対策の改善を発表しました。高度なAIを専門領域で使う際には、能力向上だけでなく、危険な利用を抑える評価やガードレールが重要だという流れが続いています。

Embeddingとの関係は少し間接的ですが、実務では大切です。意味検索は便利だからこそ、危険な手順、機微な個人情報、公開してはいけない研究資料まで、質問に近いという理由だけで出してしまう危険があります。検索品質だけでなく、「出してよい情報だけを検索対象にする」「権限で絞る」「危険な用途では人の承認を入れる」という安全設計が必要です。

今日のニュースから見える共通点は、AIが単に文章を返す段階から、データ、ツール、業務システムと接続される段階へ進んでいることです。

そして接続の最初の一步として、必要な情報を意味で探すEmbeddingの重要性は、ますます高まっています。

9. 今日の要点整理

今日のポイントを、五つに絞って振り返ります。

一つ目。Embeddingとは、文章や画像などの特徴を、多数の数値からなるベクトルに変換することです。

二つ目。ベクトルが近いほど、意味が近いとみなせることがあります。代表的な比較方法がコサイン類似度です。

三つ目。Embeddingは、答えの正しさを保証するものではありません。得意なのは、大量の情報から意味の近い候補を探すことです。

四つ目。RAGでは、文書をチャンクに分けてベクトル化し、質問に近い部分を検索してからLLMに渡します。検索、権限確認、再順位付け、根拠表示が重要です。

五つ目。実務では、Embeddingだけで完結させません。キーワード検索、日付や部署のフィルター、構造化データ、ルール、人による確認を組み合わせます。

覚え方は、こうです。

****Embeddingは、意味を座標にする。**

ベクトル検索は、近い座標を探す。

RAGは、見つけた情報を根拠に回答する。******

10. 1分理解チェック

ここから、1分理解チェックです。

少し考えてから答えを聞いてください。

第1問。

Embeddingの主な役割として、もっとも近いものはどれでしょうか。

A、文章の文字数を減らすこと。

B、文章などの特徴を数値のベクトルに変えること。

C、必ず正しい答えを生成すること。

答えは、Bです。

Embeddingは、意味の特徴を数値の並びとして扱いやすくする技術です。

第2問。

「有給休暇の申請方法」と「休暇を取りたいときの社内手続き」は、同じ単語が少なくても、Embedding検索で近い候補になり得るでしょうか。

答えは、なり得ます。

表現が違ってても、意味や用途が近ければ、近いベクトルになることがあります。

第3問。

RAGで、長い規程集を丸ごと一つのベクトルにするより、見出しや段落に沿って分割することが多いのはなぜでしょうか。

答えは、質問に関係する箇所をより正確に探し、LLMにも必要な文脈を渡しやすくするためです。

第4問。

ベクトル検索の上位結果が出たら、そのまま利用者に見せれば十分でしょうか。

答えは、十分ではありません。

閲覧権限、文書の新しさ、例外条件、検索結果の妥当性を確認する必要があります。

第5問。

「意味が近い」と「内容が正しい」は同じでしょうか。

答えは、違います。

Embeddingは関連しそうな候補を見つける助けになりますが、事実確認やルール確認の代わりにはなりません。

11. 次回予告

次回、Day 4のテーマは、「プロンプト設計の基本」です。

今日学んだEmbeddingは、必要な情報を見つけてLLMに渡す技術でした。

今回は、そのLLMに対して、どう指示すれば期待に近い回答を得られるのかを学びます。

曖昧な依頼を具体化する方法。

役割、目的、条件、出力形式を伝える方法。

そして、AIの回答を鵜呑みにせず、仕事で使える形に整える方法を扱います。

今日の帰り道か、仕事の合間に一つだけ試してみてください。

普段使っている検索で、いつものキーワードではなく、「自分が本当に達成したいこと」を文章で入力してみてください。

その検索結果を見ながら、AIは単語ではなく、意味の近さを使って候補を探しているのかもしれない、と考えてみてください。

それでは、Day 3はここまでです。おつかれさまでした。

参考情報・一次情報

1. タイトル: Previewing Ultrafast mode: GPT-5.6 Sol at up to 14X the speed

発行元: OpenAI

発表日: 2026-08-13

URL:

<https://openai.com/index/previewing-ultrafast/>

([openai.com](https://openai.com/index/previewing-ultrafast/))(

<https://openai.com/index/previewing-ultrafast/>

2. タイトル: From assistance to execution: How enterprises put AI to work

発行元: OpenAI

発表日: 2026-08-12

URL:

<https://openai.com/index/how-enterprises-put-ai-to-work/>

([openai.com](https://openai.com/index/how-enterprises-put-ai-to-work/))(

<https://openai.com/index/how-enterprises-put-ai-to-work/>

3. タイトル: Testing ads in ChatGPT

発行元: OpenAI

発表日: 2026-08-11（更新）

URL:

<https://openai.com/index/testing-ads-in-chatgpt/>

([openai.com](https://openai.com/index/testing-ads-in-chatgpt/))(

<https://openai.com/index/testing-ads-in-chatgpt/>

4. タイトル: Expanding Daybreak as the Cyber Defense Window Narrows

発行元: OpenAI

発表日: 2026-08-10

URL:

<https://openai.com/index/expanding-daybreak-as-the-cyber-defense-window-narrows/>

([openai.com](https://openai.com/index/expanding-daybreak-as-the-cyber-defense-window-narrows/))(

<https://openai.com/index/expanding-daybreak-as-the-cyber-defense-window-narrows/>

5. タイトル: Improving Fable 5 Safeguards

発行元: Anthropic

発表日: 2026-08-07

URL:

<https://www.anthropic.com/news/improving-fable-5-s-biology-safeguards>

([anthropic.com])(

<https://www.anthropic.com/news/improving-fable-5-s-biology-safeguards>